Unidad 8 · Food Store en la feria: la base de la tablet y el cierre de la capa de datos

Bases de Datos II · Grupo 11 · Trabajo Práctico N.º 8 (carpeta TP 9)

Entorno de prueba. PostgreSQL 16+ sobre una copia `food_store_tp8` (`01_schema` + `04_objects` + `02_data_demo`: 18 productos, 7 pedidos, solo datos de demo) y SQLite 3.45 (`movil/foodstore_movil.db`). La evidencia completa de los casos está en `docs/evidencias/casos_2c.txt`, generada por `casos_2c.sh`.

1.a Qué datos viajan a la tablet

Viaja (`v_snapshot_movil` → `producto_cache`)	No viaja
`id_producto` → `id`	`descripcion`, `imagen`, `categoria_id` (no hacen falta para cobrar)
`nombre`	**usuario completo**: `mail`, `contrasena`, `celular`, `nombre`, `apellido`
`precio` → `precio_centavos` (entero)	`pedido`, `detalle_pedido` (historial de ventas)
`stock` → `stock_visto` (solo referencia)	
`disponible`, `eliminado` (0/1, incluye lápidas) + `fecha_copia`	

A la tablet viaja solo lo necesario para mostrar el catálogo y armar un borrador. Los datos personales no viajan: si la tablet se pierde en la feria, no expone a nadie, y el cliente queda identificado solo por `usuario_id`. **La fuente de verdad es PostgreSQL**: `producto_cache` es una copia con fecha que se pisa con cada instantánea, y precio, stock y ventas los decide siempre el servidor.

2.c Casos de sincronización

Preparación: `casos_2c.sh` restaura la copia del servidor, exporta la instantánea S1 y arma en la tablet cinco borradores con los precios de S1 (`movil/casos/01_borradores.sql`). Después cambia el servidor (`servidor/preparar_casos.sql`: la Napolitana pasa de 5200 a 5600 y la Empanada Roquefort se da de baja lógica), exporta la instantánea S2, la carga y corre `semaforo.sql`.

Caso	Comando	Resultado obtenido
C2	Borrador con 2 Napolitanas a 5200 + `UPDATE producto SET precio = 5600.00 WHERE id_producto = 4` → semáforo	`CONFLICTO`: *producto 4 (Napolitana) cambió de precio: visto 5200.00, actual 5600.00*. Después, `casos/02_c2_aceptar.sql` → solicitud `…c2b2` = `SIMULADO_APROBADO`
C3	Borrador con 15 Flan Casero (stock 12) → semáforo	`RECHAZADO`: *sin stock suficiente: pide 15, quedan 12*
C4	Borrador con 6 Empanadas Roquefort + `UPDATE producto SET eliminado = true WHERE id_producto = 7` → S2 → semáforo	`RECHAZADO`: *fue dado de baja*. En `v_catalogo_local`, `roquefort_en_catalogo = 0`, pero la fila 7 sigue en `producto_cache` con `eliminado = 1`
C5	Borrador con 1 Muzzarella + 2 Aguas, sin cambios en el servidor → semáforo	`SIMULADO_APROBADO`: *coincide con la instantánea… (pendiente de confirmación real del servidor)*
C6	`INSERT INTO cola_sync … VALUES ('5c600000-…-00000000c600', …)` (UUID repetido) → semáforo otra vez	`UNIQUE constraint failed: cola_sync.solicitud_uuid`. La cantidad de solicitudes sigue en 1, `evaluada_en` no cambia y el semáforo informa 0 evaluadas

Explicación de cada caso.

- **C2.** El borrador guarda el precio que aceptó el cliente. Si el servidor cambió el precio, la tablet no puede decidir por el cliente: la solicitud queda en amarillo. Cuando el cliente acepta, se copia el precio nuevo de `producto_cache`, nunca al revés, y se genera una solicitud nueva. La vieja queda en `CONFLICTO` como historial.
- **C3.** `stock_visto` es una referencia y no una reserva: el vendedor puede anotar 15. El semáforo compara contra el stock de la instantánea y lo rechaza.
- **C4.** La baja lógica viaja como lápida. El UPSERT de `cargar_snapshot.sql` no borra filas, así que la FK del borrador viejo sigue siendo válida, pero el catálogo ya no muestra el producto.
- **C5.** Todo coincide, así que la luz es verde. **`SIMULADO_APROBADO` no es una venta**: el pedido real lo crea el servidor, si corresponde, cuando reciba la solicitud.
- **C6.** El UUID de la solicitud es la clave de idempotencia, así que un reintento no duplica nada. Además, el semáforo solo toma solicitudes `PENDIENTE`, por lo que repetirlo no reevalúa. **En un sistema real, la protección tiene que estar en el servidor (parada 8):** una tabla `solicitud_procesada(solicitud_uuid PRIMARY KEY, id_pedido)` donde se inserta el UUID dentro de la misma transacción de `sp_crear_pedido`. Si el UUID ya existe, se devuelve el pedido ya creado en lugar de crear otro. La PK de la tablet solo protege a esa tablet.

PostgreSQL no cambió. `servidor/verificar_servidor.sql`, antes y después de toda la simulación:

Momento	pedidos	detalles	stock_total	huella_stock (md5 de `id:stock`)
Antes (paso 0)	7	16	709	`4885fa3d84f2e2cf3b8fd536248918c7`
Después (paso 7)	7	16	709	`4885fa3d84f2e2cf3b8fd536248918c7`

Los únicos cambios en el servidor son los de **preparación** de C2 y C4 (precio y baja lógica, hechos a mano sobre la copia). Ni la tablet ni el semáforo escriben en PostgreSQL.

3.a Mapa de la capa de datos

Motor	Qué guarda	Quién manda
PostgreSQL (sistema de registro)	`categoria`, `producto` (precio, stock, disponible, baja lógica), `usuario`, `pedido`, `detalle_pedido`. Las reglas viven en CHECK, FK, triggers de subtotal y total, y `sp_crear_pedido` con `FOR UPDATE`	**Manda sobre todo.** Es la única fuente de verdad de precio, stock, usuarios y ventas
MongoDB (Unidad 7, colección `pedido`)	Un documento por pedido, con las líneas y una copia del cliente embebidas. El validador replica R1 (subtotal) y R2 (total)	**No manda**: es un modelo de lectura derivado de PostgreSQL (historial del pedido completo sin JOIN). Si hay diferencia, gana PostgreSQL y el documento se regenera
SQLite (tablet, Unidad 8)	`producto_cache` y `snapshot_servidor` (copias con fecha) · `pedido_borrador`, `item_borrador`, `cola_sync` (propuestas)	En las copias manda PostgreSQL y se pisan con cada instantánea. Los borradores son de la tablet hasta que se envían, pero **la venta la decide PostgreSQL**. La tablet nunca escribe en el servidor

Flujo: `PostgreSQL → (CSV de v_snapshot_movil) → SQLite` para el catálogo, y `SQLite → (cola_sync) → PostgreSQL` para las propuestas. PostgreSQL también alimenta a MongoDB, que es solo de lectura.

3.b Plan de mantenimiento (agregado al plan de la Unidad 6)

- **Qué se respalda y cómo se prueba.** PostgreSQL se respalda como en la Unidad 6: `pg_dump -Fc` diario (RPO 24 h) y `restore_test.sh` semanal sobre `foodstore_restore_test` (RTO 15 min). A `validacion_restore.sql` se le suman los controles de `servidor/validacion_restore_extra.sql` (H4). MongoDB no se respalda aparte, porque se regenera desde PostgreSQL con `foodstore.mongodb.js`. De la

tablet solo importa lo que todavía no llegó al servidor: al cierre de cada jornada de feria se copia el archivo con `sqlite3 foodstore_movil.db ".backup feria_AAAAMMDD.db"`. `producto_cache` no se respalda, porque se reconstruye desde la instantánea. Para probar la restauración, se abre la copia y se corre `PRAGMA integrity_check;` y `PRAGMA foreign_key_check;`, y se cuentan los borradores y solicitudes `PENDIENTE` contra los de la tablet.

- **Si se restaura un respaldo viejo de PostgreSQL.** Las tablets pueden tener una instantánea **más nueva** que el servidor restaurado, con precios o bajas que ya no existen. Por eso, ante un aviso de restauración, cada tablet descarta su catálogo, importa una instantánea nueva del servidor restaurado y vuelve a `PENDIENTE` las solicitudes evaluadas después de la fecha del respaldo, para pasarlas otra vez por el semáforo contra la verdad actual. Los pedidos confirmados después del respaldo se perdieron en el servidor. Las tablets los reenvían con **el mismo** `solicitud_uuid`, así que no se duplican los que sí hubieran sobrevivido.
- **Quién revisa y cada cuánto.** Los `CONFLICTO` los resuelve **el vendedor en el momento**, con el cliente presente, que acepta el precio nuevo o cancela. Al **cierre de cada jornada**, el encargado del food truck revisa los `RECHAZADO` y los `CONFLICTO` que quedaron abiertos. **Una vez por semana**, junto con el `restore_test`, el administrador de la base revisa los rechazos que se repiten (por ejemplo, un producto siempre sin stock).

3.c Code review con IA

Revisión: asistente de IA Claude (Anthropic, Opus 5.5), en modo de solo lectura y planificación (el mismo rol que el agente Plan de OpenCode), sobre todos los scripts de la cursada: `parcial_BD/`, `Unidad 2` a `Unidad 7` y esta `Unidad 8`. Prompt: *"Revisá todos los scripts SQL y JS de la cursada y buscá errores de lógica, reglas de negocio que no se cumplen, riesgos de concurrencia o idempotencia y controles que nunca pueden fallar; para cada uno, cómo reproducirlo en una copia y qué cambiarías"*. Cada hallazgo se reprodujo en una **copia descartable** (`createdb -T food_store_tp8 food_store_review`, que se borra al final).

#	Unidad / archivo	Hallazgo	Decisión	Motivo y comprobación en la base
H1	TPI / Unidad 2 · `parcial_BD/04_objects.sql` (`sp_crear_pedido`)	Al vender las últimas unidades, el stock queda en 0 con `disponible = true` y se viola `ck_producto_disponible_stock`: **no se pueden vender las últimas unidades**	**Corregido** (`servidor/fix_sp_crear_pedido.sql`)	`CALL sp_crear_pedido(1,'EFECTIVO','[{"producto_id":16,"cantidad":10}]')` antes del arreglo: *violates check constraint* y el pedido no se crea. Con el arreglo (`disponible = (stock - v_cantidad) > 0` en el mismo UPDATE): stock 0, `disponible = f`, y las ventas normales siguen igual
H2	TPI / Unidad 2 · `sp_crear_pedido`	No tiene clave de idempotencia: un reintento crea otro pedido y descuenta stock dos veces	**Aceptado** (mejora propuesta, no implementada en este TP)	Dos `CALL` iguales: los pedidos pasan de 7 a 9 y el stock de la Muzzarella baja 2. Es la protección de la parada 8 que se describe en C6 (`solicitud_procesada`)
H3	Unidad 8 · `movil/semaforo.sql`	Dos solicitudes pendientes de 8 Tartas cada una (stock 10) salen las dos `SIMULADO_APROBADO`. Arreglo posible: acumular en la tablet el stock pedido entre solicitudes	**Descartado** (no se corrige)	Reproducido: las dos salen en verde. El arreglo se descarta porque `SIMULADO_APROBADO` no es una venta ni una reserva, y la tablet solo ve su propia cola: otras tablets y la web también consumen stock. Acumular daría una falsa garantía. El control real es el `FOR UPDATE` de `sp_crear_pedido` en el servidor, que rechaza la segunda (*stock insuficiente*)
H4	Unidad 6 · `TP7/sql/validacion_restore.sql`	El control `precio < 0 OR stock < 0` siempre da 0, porque ya lo garantizan los CHECK. No valida las reglas que mantienen los triggers	**Corregido** (`servidor/validacion_restore_extra.sql`)	Sobre la base sana da `pedidos_total_incoherente = 0` y `detalles_subtotal_incoherente = 0`. En una copia con `UPDATE pedido SET total = total + 1 WHERE id_pedido = 1`, detecta 1